Machine Learning e Data Mining

anno accademico 2026/2027

Ambra Demontis, Fabio Brau

Pattern Recognition and Applications Lab
Department of Electrical and Electronic Engineering
University of Cagliari (Italy)



Introduzione

Machine Learning

Machine Learning

Il machine learning studia come dare ai calcolatori la capacità di apprendere dai dati, senza essere programmati in modo esplicito per il compito da risolvere.

Perché il machine learning?

Alcuni problemi è complicato descriverli con un algoritmo esplicito:

- riconoscere un volto o un gatto in un'immagine;
- capire se un'email è spam;
- prevedere il prezzo di un immobile;

Perché il machine learning?

Alcuni problemi è complicato descriverli con un algoritmo esplicito:

- riconoscere un volto o un gatto in un'immagine;
- capire se un'email è spam;
- prevedere il prezzo di un immobile;

Scrivere le regole "a mano" è:

- difficile: quali regole descrivono un gatto? Quante ne servono?
- fragile: le eccezioni sono infinite;
- costoso da mantenere: se il fenomeno cambia (nuovo spam), le regole vanno riscritte.

Perché il machine learning?

Alcuni problemi è complicato descriverli con un algoritmo esplicito:

- riconoscere un volto o un gatto in un'immagine;
- capire se un'email è spam;
- prevedere il prezzo di un immobile;

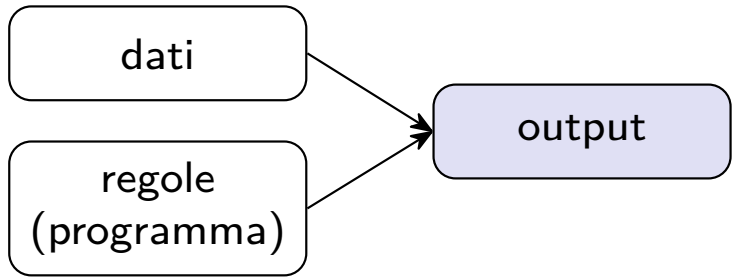
Scrivere le regole "a mano" è:

- difficile: quali regole descrivono un gatto? Quante ne servono?
- fragile: le eccezioni sono infinite;
- costoso da mantenere: se il fenomeno cambia (nuovo spam), le regole vanno riscritte.

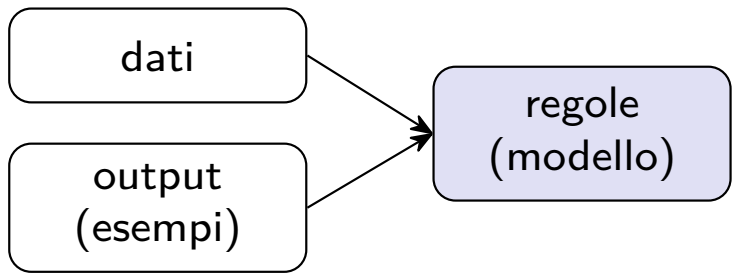
Però abbiamo spesso tanti esempi del fenomeno: email già classificate, immagini già annotate, storici di vendita...

Cambio di paradigma

Programmazione tradizionale



Machine learning



Invece di programmare la soluzione, programmiamo un procedimento che ricava la soluzione dai dati.

Il prodotto dell'apprendimento: il modello

Il Machine Learning produce una rappresentazione appresa dai dati che può essere utilizzata per fare previsioni, prendere decisioni o individuare strutture nei dati. Lo chiamiamo modello. Ad esempio una funzione parametrica

$$f_{\boldsymbol{\theta}} : \mathcal{X} \longrightarrow \mathcal{Y}, \qquad \boldsymbol{\theta} \in \mathbb{R}^M \text{ (parametri)}.$$

- $\mathcal{X}$: spazio degli ingressi (le features);
- $\mathcal{Y}$: spazio delle uscite (ciò che voglio prevedere);
- $\boldsymbol{\theta}$: i parametri, che vengono *appresi* dai dati.

Esempio: Apprendimento per rinforzo

Un agente impara a svolgere un compito interagendo con un ambiente.



- Esempi: un robot che impara a camminare, un programma che impara a giocare, la gestione del traffico in una rete.
- Non esiste una base di dati da esplorare: i dati vengono generati dall'agente stesso, provando e sbagliando.
- Il prodotto è un comportamento che massimizza la ricompensa.

Data Mining

Definizione di Data Mining

Il data mining è il processo di estrazione di schemi (*pattern*) da grandi quantità di dati; schemi che siano validi, nuovi, potenzialmente utili e comprensibili per chi deve usarli.

Esempio: Regole di Associazione

Da milioni di scontrini si estraggono regole della forma

$$\{\text{pane},\ \text{prosciutto}\} \Rightarrow \{\text{birra}\}.$$

- L'algoritmo conta le combinazioni di prodotti che ricorrono spesso insieme e ne misura l'affidabilità.
- Non c'è addestramento, né parametri da stimare, né un errore su dati nuovi da minimizzare.

Il prodotto finale è una lista di regole leggibili, consegnata a chi decide la disposizione degli scaffali o le promozioni: conoscenza, non un sistema che apprende.

Machine Learning e Data Mining

Focus differente.

Data mining: estrazione di conoscenza dai dati.
Machine learning: procedimento appreso dai dati per svolgere un compito.

La maggior parte delle tecniche si trovano nell'intersezione fra le due aree.

Dati, campioni e features

I dati

- Un campione (o esempio, istanza, osservazione) è l'oggetto che vogliamo analizzare: un'email, un paziente, un appartamento.
- Lo rappresentiamo con un vettore di features (attributi, variabili, caratteristiche)

$$\boldsymbol{x}_n = \left(x_{n1}, x_{n2}, \ldots, x_{nD}\right)^\top \in \mathbb{R}^D.$$

- L'insieme dei campioni forma la matrice dei dati (*design matrix*) $\boldsymbol{X} \in \mathbb{R}^{N \times D}$: una riga per campione, una colonna per feature.
- Se disponibile, a ogni campione è associata un'etichetta (o target, risposta) y_n.

Notazione del corso

Insieme di addestramento: $\mathcal{D} = \{(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_N, y_N)\}$, con $N =$ numero di campioni e $D =$ numero di features.

Un dataset

Prevedere il prezzo di vendita di un appartamento:

features $\boldsymbol{x}_n \in \mathbb{R}^4$				etichetta y_n
superficie [m^2]	stanze	piano	dist. centro [km]	prezzo [k€]
55	2	1	4.2	135
90	3	3	1.1	290
120	4	0	6.8	210
70	2	5	2.0	205
	⋮			⋮

- Qui $D = 4$ features numeriche e l'etichetta è un numero reale.
- Le features sono una scelta di modellazione (perché non l'anno di costruzione? l'esposizione?).
- Attributi di tipo diverso: numerici, categorici (nominali o ordinali), testo, immagini, richiedono rappresentazioni diverse.

Tipologie di apprendimento

Le principali tipologie di apprendimento

La distinzione fondamentale riguarda i dati:

- Apprendimento supervisionato: i dati sono coppie (campione, etichetta). Qualcuno ci ha detto la risposta corretta.
  - classificazione: etichette discrete;
  - regressione: etichette continue.
- Apprendimento non supervisionato: abbiamo solo i campioni, nessuna etichetta. Vogliamo scoprire la struttura dei dati.

Esistono situazioni intermedie o diverse:

- semi-supervisionato: pochi dati etichettati, molti no;
- auto-supervisionato: le etichette si costruiscono dai dati stessi;
- per rinforzo: un agente impara da ricompense ottenute interagendo con un ambiente.

Apprendimento supervisionato

Impostazione del problema

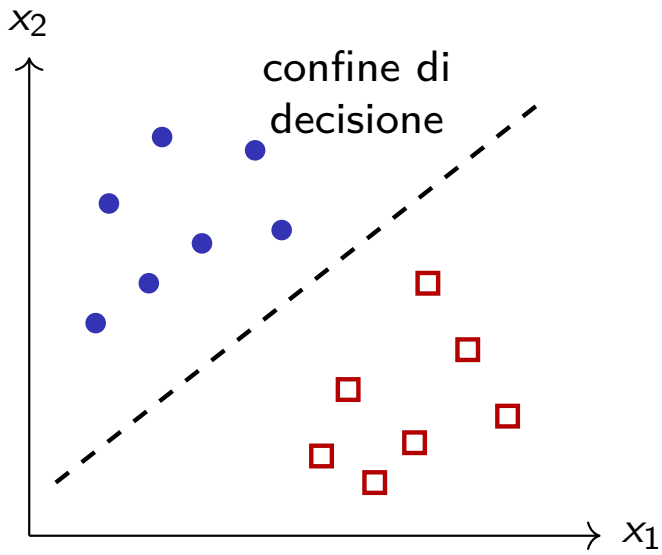
Dato $\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}_{n=1}^{N}$, con $\boldsymbol{x}_n \in \mathcal{X}$ e $y_n \in \mathcal{Y}$, cerchiamo $f : \mathcal{X} \to \mathcal{Y}$ tale che $f(\boldsymbol{x}) \approx y$ anche per campioni mai visti.

- Il "supervisore" è chi ha fornito le etichette: un medico, un utente, una misura strumentale, un processo storico.
- Le etichette sono la risorsa costosa: annotare dati richiede tempo, competenza, a volte denaro.
- La natura di $\mathcal{Y}$ determina il tipo di problema:

$\mathcal{Y}$ insieme finito di classi	$\to$ classificazione
$\mathcal{Y} = \mathbb{R}$ (o $\mathbb{R}^K$)	$\to$ regressione

Classificazione

Predire un'etichetta discreta: $\mathcal{Y} = \{c_1, \ldots, c_K\}$.

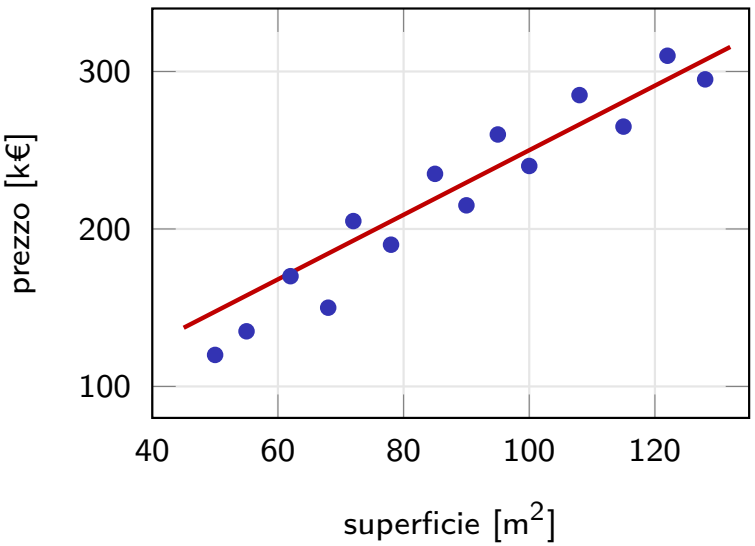


- Il modello divide lo spazio delle features in regioni di decisione, una per classe.
- Casi: binaria ($K = 2$), multi-classe ($K > 2$), multi-etichetta.

Esempi: spam/legittima; diagnosi (malato/sano); riconoscimento di cifre e caratteri; identificazione di malware; analisi del sentimento.

Regressione

Predire un valore continuo: $\mathcal{Y} = \mathbb{R}$.



- Il modello è una superficie (qui una retta) che approssima la relazione tra features ed etichetta.
- L'errore su un campione non è "giusto/sbagliato", ma una distanza: $y_n - f(\boldsymbol{x}_n)$.

Esempi: prezzo di un immobile; consumo energetico di domani; tempo di percorrenza; dose di un farmaco; età stimata da un'immagine.

Classificazione e regressione a confronto

	Classificazione	Regressione
Uscita $\mathcal{Y}$	insieme finito di classi	valori reali
Domanda	"quale categoria?"	"quanto?"
Modello	confini di decisione	funzione approssimante
Perdita tipica	errore di classificazione (giusto / sbagliato)	errore quadratico $(y - f(\boldsymbol{x}))^2$

Osservazione

Molti modelli si adattano a entrambi i casi cambiando la funzione di perdita e il modo di leggere l'uscita.

Apprendimento non supervisionato

Abbiamo solo $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$: nessuna etichetta.

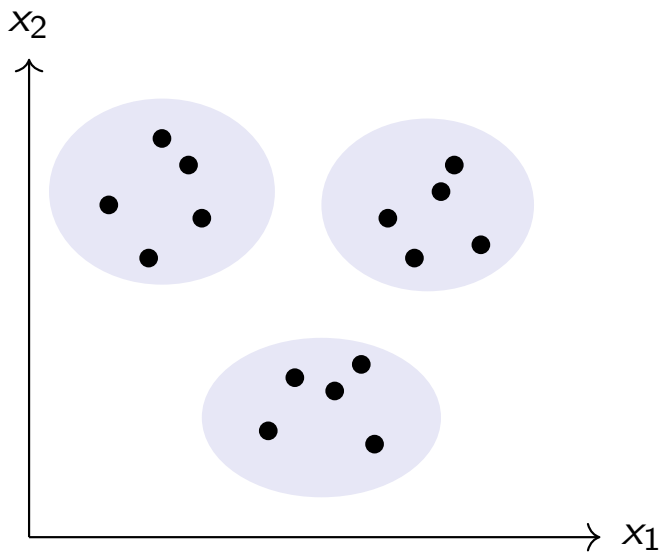
Non c'è una risposta corretta da imitare; l'obiettivo è descrivere i dati. I problemi tipici:

- clustering: raggruppare i campioni simili tra loro;
- riduzione della dimensionalità: rappresentare i dati con meno features, perdendo poca informazione;
- rilevazione di anomalie: capire dove i dati si addensano, per riconoscere i campioni atipici;
- regole di associazione: co-occorrenze frequenti.

Difficoltà

Senza etichette, valutare il risultato è intrinsecamente più ambiguo: quanti gruppi ci sono? Un raggruppamento è "buono" o solo plausibile?

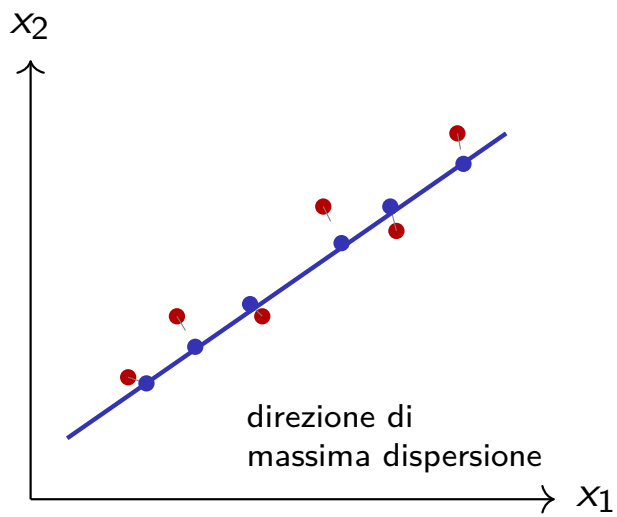
Esempio: clustering



- Cerchiamo gruppi (*cluster*) di campioni simili, secondo una nozione di distanza.
- I gruppi non hanno un nome: l'interpretazione è dell'esperto.
- Esempi: segmentazione della clientela; raggruppamento di documenti o notizie; individuazione di famiglie di malware.

Attenzione: cambiando la definizione di "simile" (o la scala delle features!) cambiano i gruppi trovati.

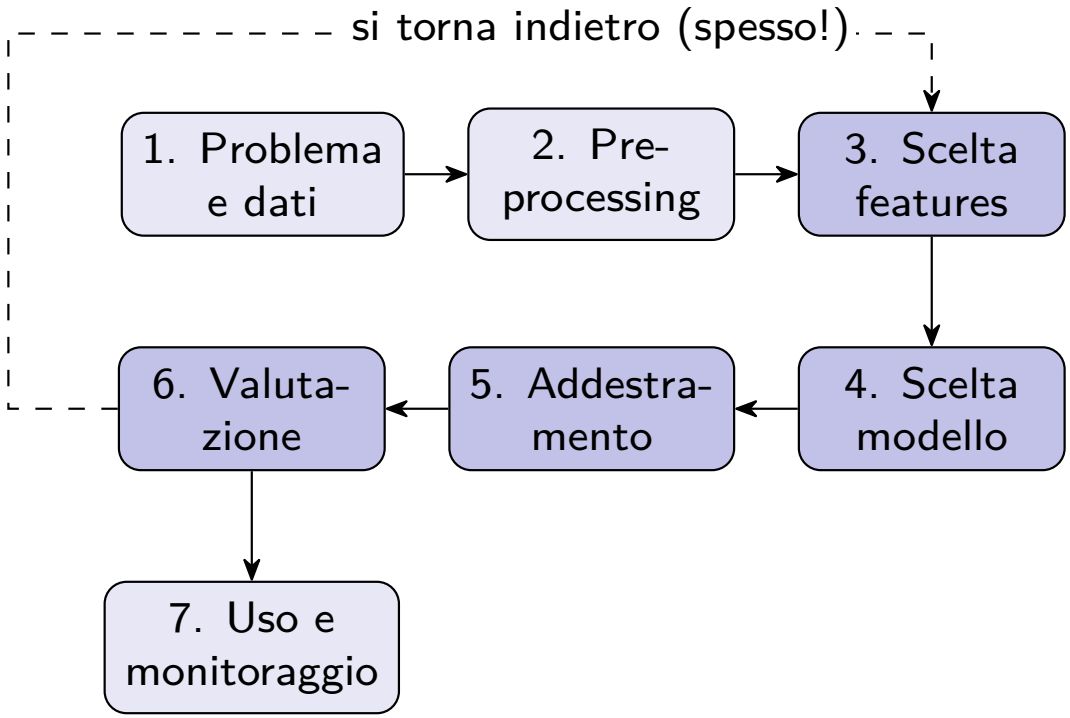
Esempio: riduzione della dimensionalità



- Molte features sono ridondanti o correlate: i dati vivono (quasi) in un sottospazio di dimensione minore.
- Proiettandoli su quel sottospazio conserviamo l'informazione utile con meno numeri (es. PCA).
- A cosa serve: visualizzare, comprimere, rimuovere rumore, preparare i dati per un modello supervisionato.

Apprendimento supervisionato

Le fasi



- Non è una sequenza lineare: è un ciclo.

1–2. Dati e pre-processing

- Definire il problema: che cosa voglio prevedere, con quali informazioni, e quanto costa sbagliare (in un verso o nell'altro).
- Raccogliere i dati: quantità, qualità e rappresentatività rispetto ai casi d'uso reali.
- Pulizia e preparazione:
  - valori mancanti, duplicati, outlier, errori di misura;
  - codifica delle variabili categoriche;
  - normalizzazione / standardizzazione delle scale;
  - classi sbilanciate.

3. Scelta delle features

Una feature è una misura del campione che riteniamo informativa per il compito.

- Feature engineering: costruirle usando la conoscenza del dominio (es. prezzo per m^2, rapporti, differenze, medie mobili).
- Selezione: scartare features irrilevanti o ridondanti.
- Estrazione: costruire nuove features come combinazione delle originali (es. PCA).
- Feature learning: lasciare che sia il modello a costruirle (è ciò che fanno le reti profonde).

3. Scelta delle features

Una feature è una misura del campione che riteniamo informativa per il compito.

- Feature engineering: costruirle usando la conoscenza del dominio (es. prezzo per m^2, rapporti, differenze, medie mobili).
- Selezione: scartare features irrilevanti o ridondanti.
- Estrazione: costruire nuove features come combinazione delle originali (es. PCA).
- Feature learning: lasciare che sia il modello a costruirle (è ciò che fanno le reti profonde).

Più features non significa modello migliore:

- all'aumentare di D lo spazio si "svuota" e servono esponenzialmente più dati (maledizione della dimensionalità);
- features rumorose o irrilevanti facilitano l'overfitting.

4. Scelta del modello

Scegliere un modello significa scegliere una famiglia di funzioni candidate $\mathcal{H}$ (spazio delle ipotesi):

$$\mathcal{H} = \{ f_{\boldsymbol{\theta}} : \boldsymbol{\theta} \in \mathbb{R}^M \}.$$

- Esempi: funzioni lineari; polinomi di grado p; alberi di decisione; reti neurali.
- La complessità (o capacità) di $\mathcal{H}$ è cruciale:
  - troppo semplice $\rightarrow$ non riesce a descrivere i dati;
  - troppo complessa $\rightarrow$ descrive anche il rumore.
- Le scelte non apprese dai dati (grado del polinomio, intensità della regolarizzazione, ...) si chiamano iperparametri.

4. Scelta del modello

Scegliere un modello significa scegliere una famiglia di funzioni candidate $\mathcal{H}$ (spazio delle ipotesi):

$$\mathcal{H} = \{ f_{\boldsymbol{\theta}} : \boldsymbol{\theta} \in \mathbb{R}^M \}.$$

- Esempi: funzioni lineari; polinomi di grado p; alberi di decisione; reti neurali.
- La complessità (o capacità) di $\mathcal{H}$ è cruciale:
  - troppo semplice $\rightarrow$ non riesce a descrivere i dati;
  - troppo complessa $\rightarrow$ descrive anche il rumore.
- Le scelte non apprese dai dati (grado del polinomio, intensità della regolarizzazione, ...) si chiamano iperparametri.

Nessun modello è il migliore sempre

La scelta va guidata da: natura del problema, quantità di dati disponibili, requisiti di interpretabilità e costo computazionale.

5. Addestramento

Fissata la famiglia $\mathcal{H}$, serve un criterio per scegliere $\boldsymbol{\theta}$. Definiamo una funzione di perdita $L(y, f_{\boldsymbol{\theta}}(\boldsymbol{x}))$, che misura quanto sbagliamo su un singolo campione, e minimizziamo l'errore medio sui dati di addestramento (*rischio empirico*):

$$\boldsymbol{\theta}^{\star} = \arg\min_{\boldsymbol{\theta}} \; \frac{1}{N} \sum_{n=1}^{N} L(y_n,\, f_{\boldsymbol{\theta}}(\boldsymbol{x}_n)) \; + \underbrace{\lambda\, \Omega(\boldsymbol{\theta})}_{\text{regolarizzazione}} \quad .$$

- Apprendere diventa quindi un problema di ottimizzazione, risolto con una formula chiusa o iterativamente (discesa del gradiente).
- Il termine di regolarizzazione penalizza le soluzioni "troppo complesse".

6. Valutazione delle prestazioni

Le prestazioni non si misurano sui dati usati per addestrare.



- Training set: stima dei parametri $\boldsymbol{\theta}$.
- Validation set: scelta di iperparametri e confronto fra modelli. Con pochi dati si usa la k-fold cross-validation.
- Test set: stima finale e imparziale dell'errore. Si usa una volta sola, alla fine.
- Attenzione al data leakage: ogni scelta fatta guardando il test set ne invalida la stima (anche la normalizzazione!).

Generalizzazione

L'obiettivo: generalizzare

Durante l'addestramento sappiamo calcolare solo l'errore sui dati che abbiamo:

$$E_{\text{train}}(f) = \frac{1}{N} \sum_{n=1}^{N} L(y_n, f(\boldsymbol{x}_n)).$$

Quello che ci interessa è invece l'errore medio sui casi futuri, che per definizione non possiamo calcolare: quei dati non li abbiamo ancora.

- Lo stimiamo mettendo da parte una porzione di dati (il test set) e trattandola come se fosse "il futuro".
- Ipotesi implicita: i dati futuri nascono dallo stesso fenomeno, raccolto nello stesso modo. Se le condizioni cambiano (nuovi utenti, nuovo tipo di spam), la stima non vale più.

L'obiettivo: generalizzare

Durante l'addestramento sappiamo calcolare solo l'errore sui dati che abbiamo:

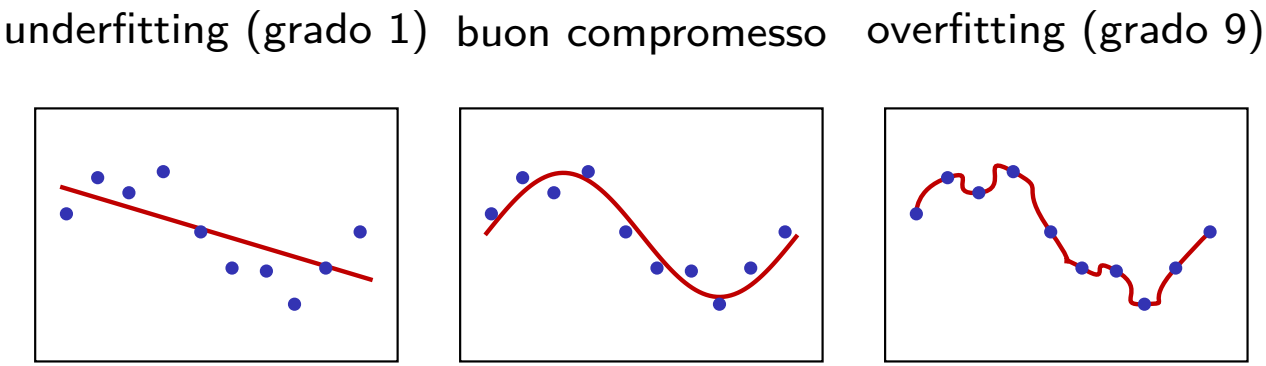
$$E_{\text{train}}(f) = \frac{1}{N} \sum_{n=1}^{N} L(y_n, f(\boldsymbol{x}_n)).$$

Quello che ci interessa è invece l'errore medio sui casi futuri, che per definizione non possiamo calcolare: quei dati non li abbiamo ancora.

- Lo stimiamo mettendo da parte una porzione di dati (il test set) e trattandola come se fosse "il futuro".
- Ipotesi implicita: i dati futuri nascono dallo stesso fenomeno, raccolto nello stesso modo. Se le condizioni cambiano (nuovi utenti, nuovo tipo di spam), la stima non vale più.

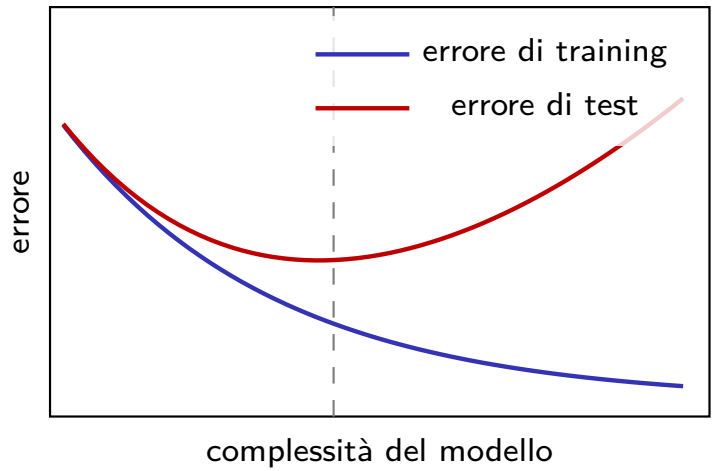
Memorizzare non è apprendere.

Underfitting e overfitting



- Underfitting: il modello è troppo semplice, sbaglia sia in addestramento sia in test.
- Overfitting: il modello segue anche il rumore dei dati di addestramento; errore piccolissimo in training, grande in test.

Il compromesso tra complessità ed errore



- L'errore di training diminuisce sempre aumentando la complessità.
- L'errore di test ha tipicamente un minimo: è lì che vogliamo stare.
- A sinistra il modello è troppo rigido; a destra troppo sensibile ai dettagli (e al rumore) dei dati di addestramento.

La complessità "giusta" non si legge nei dati di addestramento: va scelta sulla validazione.

Come si contrasta l'overfitting

- Più dati (o dati più rappresentativi): è quasi sempre la soluzione più efficace.
- Ridurre la complessità: meno features, modello più semplice.
- Regolarizzazione: penalizzare parametri grandi ($\lambda \|\boldsymbol{\theta}\|^2$).

Non cerchiamo il modello che descrive meglio i dati che abbiamo, ma quello che prevede meglio i dati che non abbiamo ancora visto.